Java Persistence API

Ehsan Khamespanah

Spring 2026

Data Persistence and Domain Objects

- Business code access DB tables rather than objects!
- Entities are based on database structure rather than business concepts.
- Transaction management and automatic key generation is not easy.
- It is hard to maintain SQL queries as there is no refactoring mechanism

Data Persistence and Domain Objects

- Business code access DB tables rather than objects!
- Entities are based on database structure rather than business concepts.
- Transaction management and automatic key genera
- It is ha ho refacto

Object Oriented Database
v.s.
Object Relational Mapping

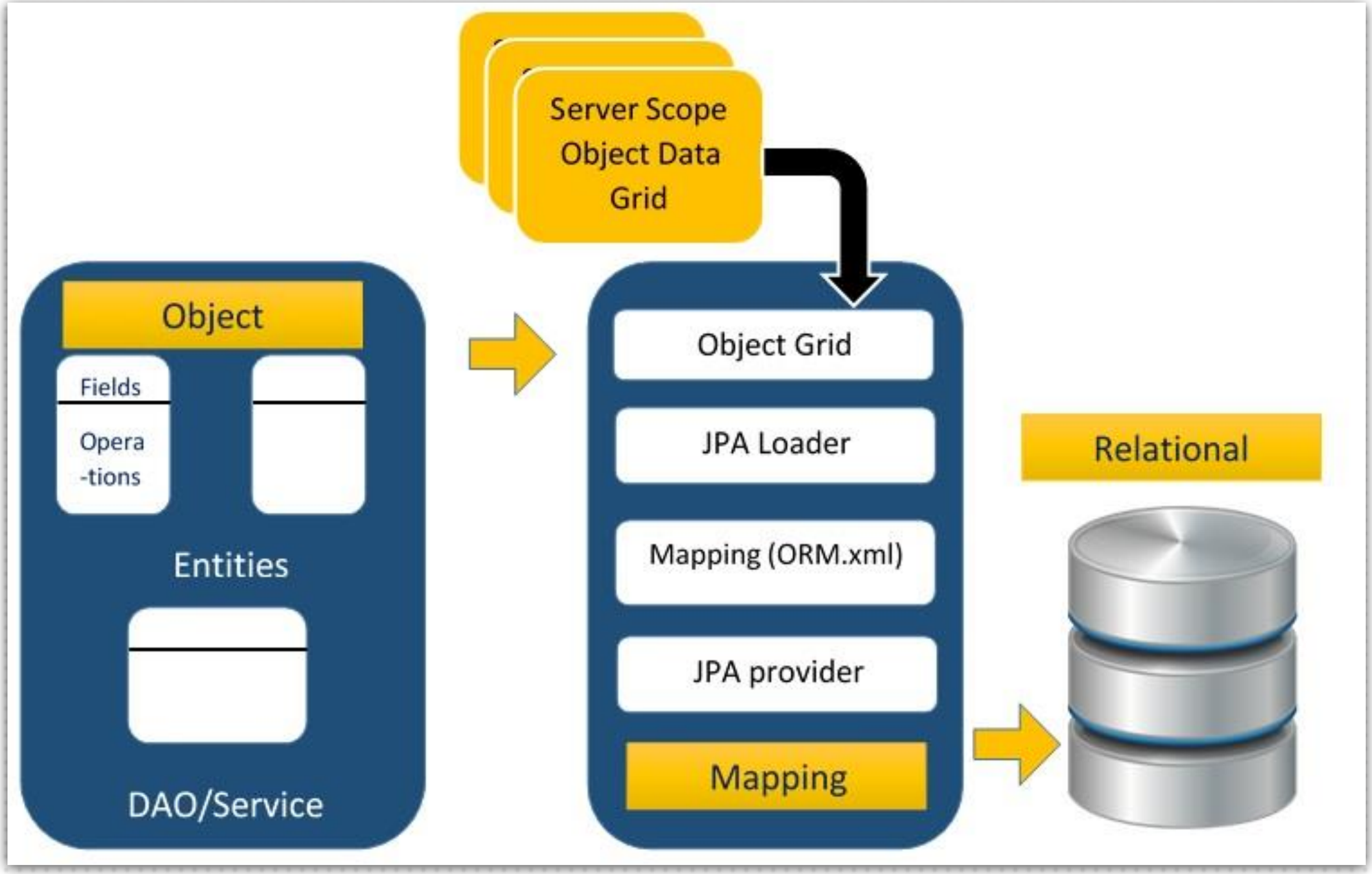
ORM Advantages

- Lets business code access objects rather than DB tables.
- Hides details of SQL queries from OO logic.
- Based on JDBC 'under the hood'
- No need to deal with the database implementation.
- Entities based on business concepts rather than database structure.
- Transaction management and automatic key generation.
- Fast development of application.

ORM Solutions Provide

- An API to perform basic CRUD operations on objects of persistent classes.
- A language or API to specify queries that refer to classes and properties of classes.
- A configurable facility for specifying mapping metadata.
- A technique to interact with transactional objects to perform dirty checking, lazy association fetching, and other optimization functions.

ORM Architecture



Review The Tradition!

- The Musician class has different attributes, some of them are ordinary type fields and some are complex

```
public class Musician {
  private Long id;
  private String name;
  private Instrument mainInstrument;
  private ArrayList performances = new ArrayList<Performance>();

  public Musician( Long id, String name){ /* constructor setters... */ }

  public String getName(){
    return this.name;
  }
  public Instrument getMainInstrument(){
    return this.instrument;
  }
  // ...Other getters and setters...
}
```

Storing Data

- Storing a Musician

```
Musician georgeHarrison = new Musician(0, "George Harrison");

String myDriver = "org.gjt.mm.mysql.Driver";
String myUrl = "jdbc:mysql://localhost/test";
Class.forName(myDriver);
Connection conn = DriverManager.getConnection(myUrl, "root", "");

String query = " insert into users (id, name) values (?, ?)";
PreparedStatement preparedStmt = conn.prepareStatement(query);
preparedStmt.setInt(1, 0);
preparedStmt.setString(2, "George Harrison");
preparedStmt.setString(2, "Rubble");

preparedStmt.execute();
```

- Designing a manager

```
Musician georgeHarrison = new Musician(0, "George Harrison");
musicianManager.save(georgeHarrison);
```

ORM Helps By Defining Entities

- In order to define an entity, a class has to be annotated with the @Entity annotation

```
@Entity
public class Book {
    ...
}
```

- Custom names for the corresponding tables is allowed

```
@Entity
@Table(name="BOOKS")
public class Book {
    ...
}
```

Mapping Fields to Columns

- Fields of an entity are automatically mapped to table columns with the name name
- You can override it by @Column
- You can define table id columns

```
@Entity
@Table(name="BOOKS")
public class Book {
    @Id
    private Integer id;
    private String name;
    @Column(name="ISBN_NUMBER")
    private String isbn;
    ...
}
```

Different Strategies for ID Generation

- Persistence providers chooses a generation strategy

```
@Id
@GeneratedValue(strategy = GenerationType.AUTO)
@Column(name = "id", updatable = false, nullable = false)
private Long id;
```

- GenerationType.IDENTITY: It relies on an auto-incremented database column (performance issues)
- GeneratioType.SEQUENCE: is a database object that generates incremental integers on each successive request
- GenerationType.TABLE: Never use it!

Relationship Between Entities

- One to One relation

```
@Entity
public class User {
   @Id
   private Integer id;
   private String email;
   private String name;
   @OneToOne(mappedBy=“user")
   private UserProfile profile;
   ...
}

@Entity
public class UserProfile {
   @Id
   private Integer id;
   private int age;
   private String favoriteColor;
   @OneToOne
   private User user;
   ...
}
```

Relationship Between Entities

- One to Many and Many to One relations

```java
@Entity
public class Book {
    @Id
    private Integer id;
    private String name;
    @ManyToOne
    @JoinColumn(name="AUTHOR_ID")
    private Author author;
    ...
}

@Entity
public class Author {
    @Id
    @GeneratedValue
    private Integer id;
    private String name;
    @OneToMany(mappedBy = "author")
    private List<Book> books = new ArrayList<>();
    ...
}
```

Relationship Between Entities

- Many to Many relation

```
@Entity
public class Book {
    @Id
    private Integer id;
    private String name;
    @ManyToMany
    @JoinTable(name="BOOK_AUTHORS",
            joinColumns=@JoinColumn(name="BOOK_ID"),
            inverseJoinColumns=@JoinColumn(name="AUTHOR_ID"))
    private Set<Author> authors = new HashSet<>();
}

@Entity
public class Author {
    @Id
    @GeneratedValue
    private Integer id;
    private String name;
    @ManyToMany(mappedBy = "author")
    private Set<Book> books = new HashSet<>();
}
```

EntityManager for Working with Entities

- EntitiyManager is usually configured using and XML specification

```
<persistence xmlns=“http://java.sun.com/xml/ns/persistence" ...>
    <persistence-unit name="Books" transaction-type="RESOURCE_LOCAL">
        <!-- Persistence provider -->
        <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>
        <!-- Entity classes -->
        <class>com.geekcap.javaworld.jpa.model.Book</class>
        <class>com.geekcap.javaworld.jpa.model.Author</class>
        <properties>
            <property name="javax.persistence.jdbc.driver" value="org.h2.Driver" />
            <property name="javax.persistence.jdbc.url"    value="jdbc:h2:mem:bookstore" />
            <property name="javax.persistence.jdbc.user" value="sa" />
            <property name="javax.persistence.jdbc.password" value="" />
            <property name="hibernate.dialect" value="org.hibernate.dialect.H2Dialect"/>
           <property name="show_sql" value="true"/>
       </properties>
   </persistenc
```

```
EntityManagerFactory entityManagerFactory = Persistence.createEntityManagerFactory("Books");
EntityManager entityManager = entityManagerFactory.createEntityManager();
```

CRUD Operations

- Storing an author

```
Author author = new Author(“Ali");
author.addBook(new Book("Book 1"));
author.addBook(new Book("Book 2"));
author.addBook(new Book("Book 3"));
entityManager.getTransaction().begin();
entityManager.persist(author);
entityManager.getTransaction().commit();
```

- Updating the author info

```
entityManager.getTransaction().begin();
CriteriaBuilder builder = entityManager.getCriteriaBuilder();
CriteriaQuery<Author> criteria = builder.createQuery(Author.class);
Root<Topic> root = criteria.from(Author.class);
criteria.where(
    builder.equal(root.get("name"), "Ali")
);
Author author = entityManager.createQuery(criteria).getResultList().get(0);
author.setName(“Vali");
entityManager.getTransaction().commit();
```

CRUD Operations

- Remove an author

```
entityManager.getTransaction().begin();
//find the author
entityManager.remove(author);
entityManager.getTransaction().commit();
```

- Casecade deletion to remove orphans

```
@Entity
public class Author {
    @Id
    @GeneratedValue
    private Integer id;
    private String name;
    @OneToMany(mappedBy = “author”, cascade = CascadeType.ALL)
    private List<Book> books = new ArrayList<>();
    ...
}
```

- Does not work for the case of Many to Many relation

CRUD Operations

- Different options for retrieving data
- Id based retrieval

```
Author author = entityManager.find(Author.class, 12);
```

- Using type safe criterial builder as we did for update

- Different ways to write select queries

```
EntityManager em = entityManagerFactory.createEntityManager();
Query query = em.createQuery("SELECT e FROM Employee e WHERE e.dept = :deptName");
query.setParameter("deptName", dept);
List<Employee> resultList = query.getResultList();
```

```
select cat from Cat as cat where cat.birthdate < 12

select mother from Cat as cat join cat.mother as mother where cat.name = 'cname'

select cat from DomesticCat cat where cat.name = 'dcname'
```

The Problem Before Spring Data JPA

- For every entity, developers must: Create repository classes, inject EntityManager, write CRUD methods, and manage queries

```java
@Repository
public class StudentRepository {

    @PersistenceContext
    private EntityManager entityManager;

    public Student save(Student student) {
        entityManager.persist(student);
        return student;
    }

    public Student findById(Long id) {
        return entityManager.find(Student.class, id);
    }
}
```

- This becomes repetitive

What Spring Data JPA Does

- Spring Data JPA sits on top of JPA and Hibernate

Layer	Responsibility
Spring Data JPA	Repository abstraction
JPA	Persistence specification
Hibernate	ORM implementation
Database	Data storage

How Spring Data JPA Works

- Spring automatically creates an implementation at runtime
- It provides:
  - save()
  - findById()
  - findAll()
  - delete()
  - count()
  - existsById()

```java
@Entity
public class Student {

    @Id
    @GeneratedValue
    private Long id;

    private String name;

    private int age;
}
```

```java
public interface StudentRepository
        extends JpaRepository<Student, Long> {
}
```

```java
Student s = repository.findById(1L);

s.setName("Alice Smith");

repository.save(s);
```

Query Methods (Magic Feature)

- You only define method names; Spring generates the query automatically.

```
public interface StudentRepository
        extends JpaRepository<Student, Long> {

    List<Student> findByName(String name);
    List<Student> findByAge(int age);
    List<Student> findByAgeGreaterThan(int age);
    List<Student> findByNameContaining(String text);
    List<Student> findByNameAndAge(String name, int age);
    @Query("""
            SELECT s
            FROM Student s
            WHERE s.age > :age
           """)
    List<Student> findOlderThan(@Param("age") int age);
}
```

Fetching Strategies

- Loading all field objects upon loading a class object
  - It may load a huge number of objects in retrieving a single entity because of transitive relations between objects

```
@Entity
public class Author {
    ...
    @OneToMany(mappedBy = “author”, cascade = CascadeType.ALL,
                    fetch = FetchType.EAGER)
    private List<Book> books = new ArrayList<>();
}
```

- Load field objects later using FetchType.Lazy
  - Field objects are load in their first access

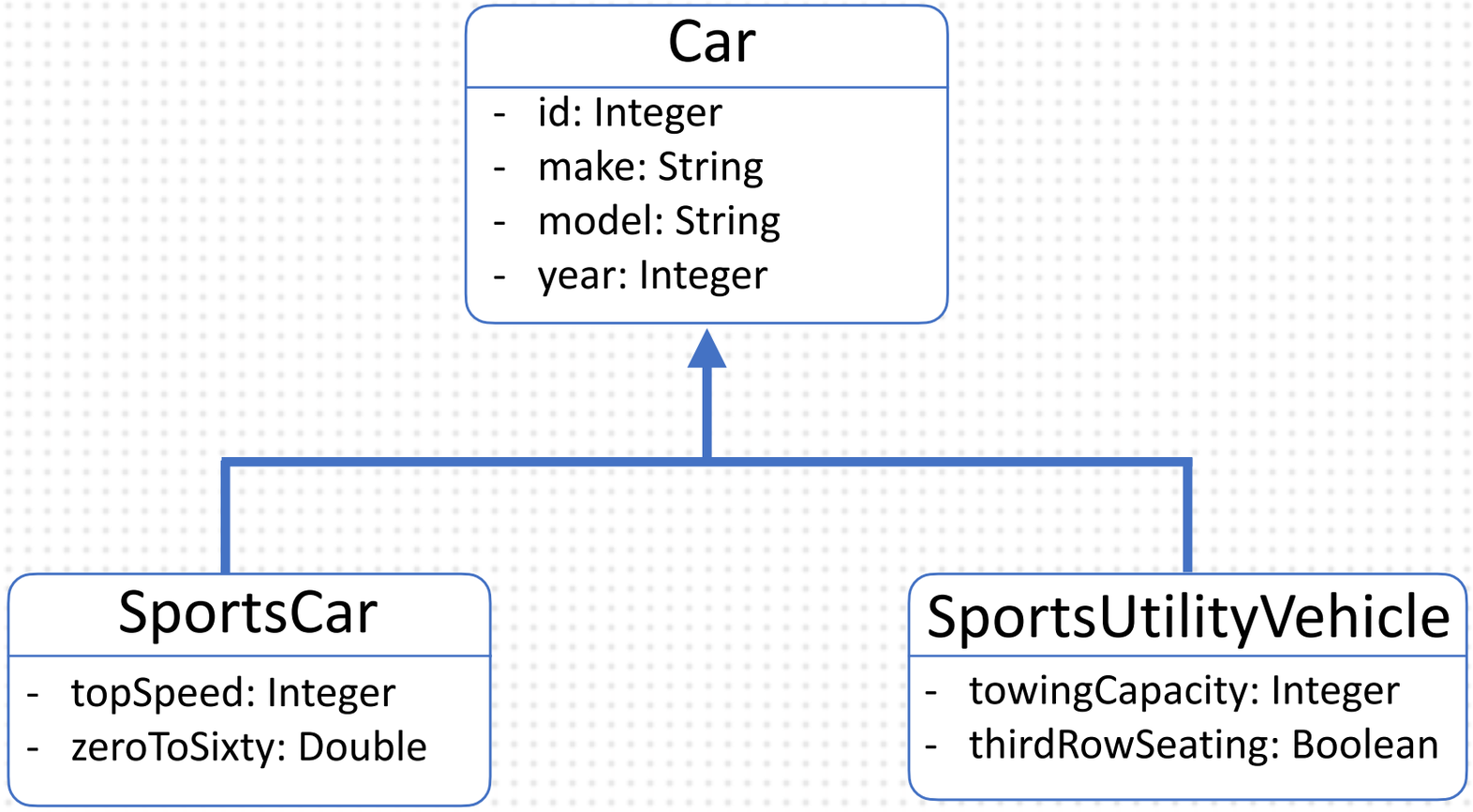
Eager Loading Pros and Cons

- Advantages:
  - No delayed initialization related performance impacts
- Disadvantages:
  - Long initial loading time
  - Loading too much unnecessary data might impact performance

Lazy Loading Pros and Cons

- Advantages:
  - Initial load time much smaller than in the other approach
  - Less memory consumption than in the other approach
- Disadvantages:
  - Delayed initialization might impact performance during unwanted moments
  - In some cases you need to handle lazily-initialized objects with a special care or you might end up with an exception (e.g. accessing to a field data after closing its corresponding transaction)

Dealing with Inheritance



- Different Strategies: Mapped Superclass, Table Per Class, Single Table, Joined

Mapped Super Class

- Two tables are created for the two concrete classes
- Each class contains the original Car attributes and specialized class attributes
- No polymorphic query: CarDealer class that manages different types of cars?

```
@MappedSuperclass
public abstract class Car {
    @Id
    @GeneratedValue
    private Integer id;
    private String make;
    private String model;
    private Integer year;
}
```

```
@Entity
@Table(name = "SPORTSCAR")
public class SportsCar extends Car {
    private Integer topSpeed;
    private Double zeroToSixty;
}
@Entity
@Table(name = "SUV")
public class SUV extends Car {
    private Integer towingCapacity;
    private Boolean thirdRowSeating;
}
```

Table per Class

- Similar to Mapped Super Class but polymorphic queries is allowed

```
@Entity
@Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)
public abstract class Car {
    @Id
    @GeneratedValue
    private Integer id;
    private String make;
    private String model;
    private Integer year;
}
```

```
Table: SPORTSCAR
  {ID: 1, MAKE: Carrera, MODEL: Porche, YEAR: 2018, TOPSPEED: 150, ZEROTOSIXTY: 4.5}
Table: SUV
  {ID: 2, MAKE: CX-9, MODEL: Mazda, YEAR: 2017, THIRDROW: true, TOWINGCAPACITY: 3000}
```

- Potential performance penalty when union is needed

Single Table

- Mapping all fields from all entities to a single table
- A discriminator column to identify the instance type
- Allows for efficient and polymorphic queries, but require maintaining null and unnecessary columns

```java
@Entity
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name = "Car_Type")
public abstract class Car {
    @Id
    @GeneratedValue
    private Integer id;
    private String make;
    private String model;
    private Integer year;
    ...
}
```

Single Table

```
@Entity
@Table(name = "SPORTSCAR")
@DiscriminatorValue("SportsCar")
public class SportsCar extends Car {
    private Integer topSpeed;
    private Double zeroToSixty;
    ...
}

@Entity
@Table(name = "SUV")
@DiscriminatorValue("SUV")
public class SportsUtilityVehicle extends Car {
    private Integer towingCapacity;
    private Boolean thirdRowSeating;
   ...
}
```

```
Table: CAR
  {CAR_TYPE: SportsCar, ID: 1, MAKE: Carrera, MODEL: Porche, YEAR: 2018, TOPSPEED: 150,
       ZEROTOSIXTY: 4.5, THIRDROW: null, TOWINGCAPACITY: null},
  {CAR_TYPE: SUV, ID: 2, MAKE: CX-9, MODEL: Mazda, YEAR: 2017, TOPSPEED: null,
       ZEROTOSIXTY: null, THIRDROW: true, TOWINGCAPACITY: 3000},
```

Joined

- Separates data between the base class and specialized classes into their own tables

```
@Entity
@Inheritance(strategy = InheritanceType.Joined)
@DiscriminatorColumn(name = "Car_Type")
public abstract class Car {
  ...
}
```

```
Table: CAR
  {ID: 1, MAKE: Carrera, MODEL: Porche, YEAR: 2018}
  {ID: 2, MAKE: CX-9, MODEL: Mazda, YEAR: 2017}
Table: SPORTSCAR
  {TOPSPEED: 150, ZEROTOSIXTY: 4.5, ID: 1}
Table: SUV
  {THIRDROW: true, TOWINGCAPACITY: 3000, ID: 2}
```

- Efficient to store data but expensive as requires Join

Which Strategy?

- Avoiding the TABLE_PER_CLASS strategy because of the query overhead.
- Use MAPPED_SUPERCLASS strategy only for cases where there is no need to use classes polymorphically.
- For fast queries and don't mind maintaining extra, unused null columns in a single table, go with the SINGLE_TABLE strategy.
- Want to model inheritance in natural way and don’t store unused columns, then use the JOINED strategy.

Cascade Type

- Entity relationships often depend on the existence of another entity
  - e.g. the Person–Address relationship
  - Without the Person, the Address entity doesn't have any meaning of its own.
  - When we delete the Person entity, our Address entity should also get deleted.
- Cascading is the way to achieve this

CascadeType.PERSIST

- Propagates the persist operation from a parent to a child entity

```
@Test
public void whenParentSavedThenChildSaved() {
    Person person = new Person();
    Address address = new Address();
    address.setPerson(person);
    person.setAddresses(Arrays.asList(address));
    session.persist(person);
    session.flush();
    session.clear();
}
```

```
Hibernate: insert into Person (name, id) values (?, ?)
Hibernate: insert into Address (
    city, houseNumber, person_id, street, zipCode, id) values (?, ?, ?, ?, ?, ?)
```

CascadeType.MERGE

- Propagates the merge operation from a parent to a child entity

```
@Test
public void whenParentSavedThenMerged() {
    int addressId;
    Person person = buildPerson("devender");
    Address address = buildAddress(person);
    person.setAddresses(Arrays.asList(address));
    session.persist(person);
    session.flush();
    addressId = address.getId();
    session.clear();

    Address savedAddressEntity = session.find(Address.class, addressId);
    Person savedPersonEntity = savedAddressEntity.getPerson();
    savedPersonEntity.setName("devender kumar");
    savedAddressEntity.setHouseNumber(24);
    session.merge(savedPersonEntity);
    session.flush();
}
```

CascadeType.MERGE

- Propagates the merge operation from a parent to a child entity

```
@Test
public void whenParentSavedThenMerged() {
    int addressId;
    Person person = buildPerson("devender");
    Address address = buildAddress(person);
    person.setAddresses(Arrays.asList(address));
    session.persist(person);
    session.flush();
    addressId = address.getId();
    session.clear();
```

```
Hibernate: select address0_.id as id1_0_0_, address0_.city as city2_0_0_,
address0_.houseNumber as houseNum3_0_0_, address0_.person_id as person_i6_0_0_,
address0_.street as street4_0_0_, address0_.zipCode as zipCode5_0_0_ from
Address address0_ where address0_.id=?
Hibernate: select person0_.id as id1_1_0_, person0_.name as name2_1_0_ from
Person person0_ where person0_.id=?
Hibernate: update Address set city=?, houseNumber=?, person_id=?, street=?,
zipCode=? where id=?
Hibernate: update Person set name=? where id=?
```

CascadeType.REMOVE

- Removes the row corresponding to the entity from the database and also from the persistent context

```
@Test
public void whenParentRemovedThenChildRemoved() {
    int personId;
    Person person = buildPerson("devender");
    Address address = buildAddress(person);
    person.setAddresses(Arrays.asList(address));
    session.persist(person);
    session.flush();
    personId = person.getId();
    session.clear();

    Person savedPersonEntity = session.find(Person.class, personId);
    session.remove(savedPersonEntity);
    session.flush();
}
```

```
Hibernate: delete from Address where id=?
Hibernate: delete from Person where id=?
```

CascadeType.REMOVE vs orphanRemoval

- Cascading remove operation is a dangerous operation

```
@Entity
public class OrderRequest {

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;

    @OneToOne(cascade = { CascadeType.REMOVE, CascadeType.PERSIST })
    private ShipmentInfo shipmentInfo;

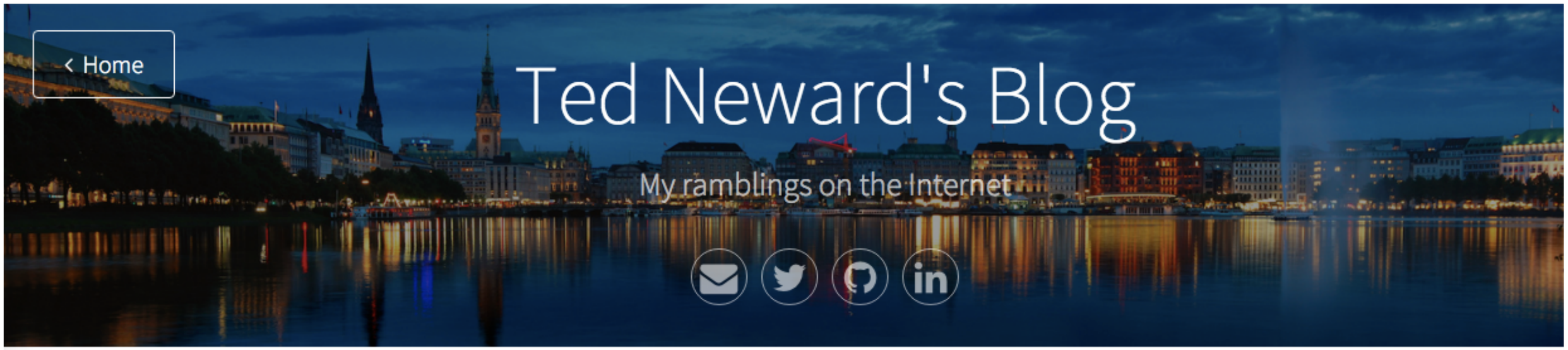
    @OneToMany(orphanRemoval = true, cascade = CascadeType.PERSIST,
       mappedBy = "orderRequest")
    private List<LineItem> lineItems;

    // constructors

    public void removeLineItem(LineItem lineItem) {
        lineItems.remove(lineItem);
    }
}
```

Other CascadeType

- CascadeType.ALL
- CascadeType.DETACH
- CascadeType.REFRESH
- CascadeType.REPLICATE
- CascadeType.SAVE_UPDATE



The Vietnam of Computer Science

Posted on Jun 26, 2006

#.NET #Java/J2EE #C++ #Ruby

(Two years ago, at Microsoft's TechEd in San Diego, I was involved in a conversation at an after-conference event with Harry Pierson and Clemens Vasters, and as is typical when the three of us get together, architectural topics were at the forefront of our discussions. An crowd gathered around us,